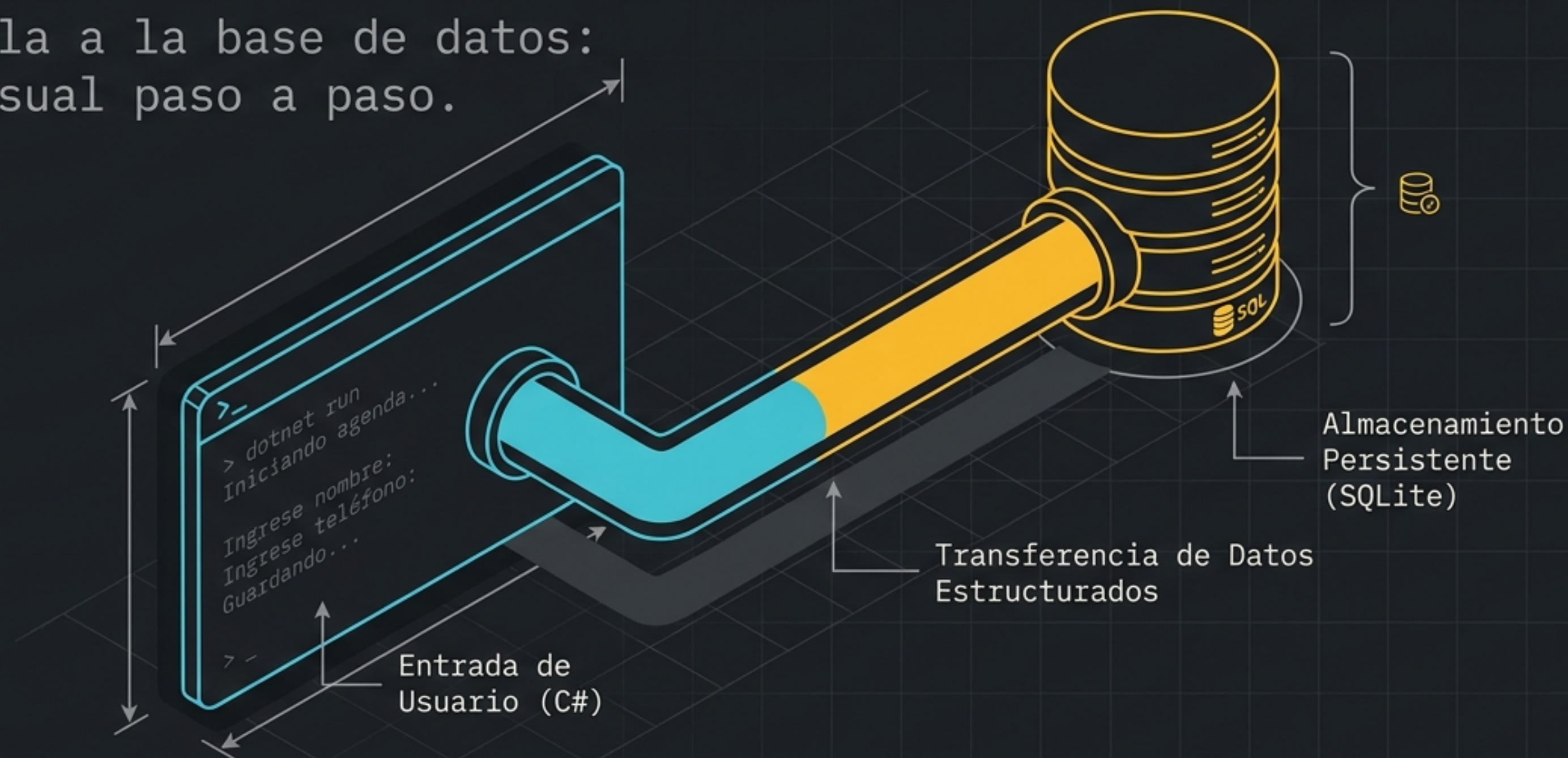
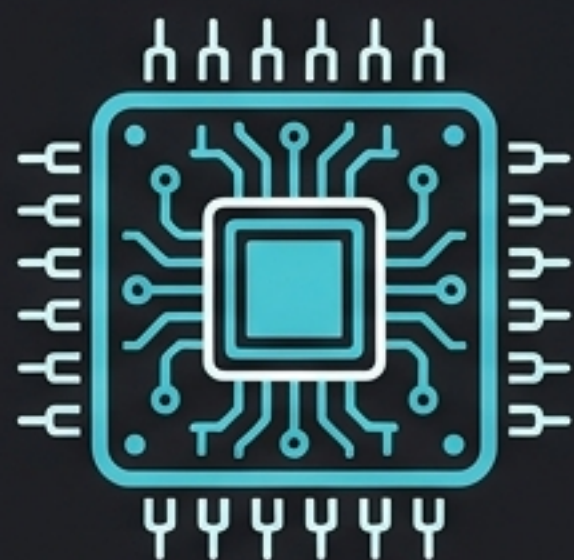


Construyendo tu Agenda: El Motor C# y SQLite

De la consola a la base de datos:
una guía visual paso a paso.



El Problema de la Memoria Volátil



Usar una `List<Contacto>` en C#

➤ Rápido, pero se borra al cerrar el programa. Cada vez que reinicias, la agenda está vacía.

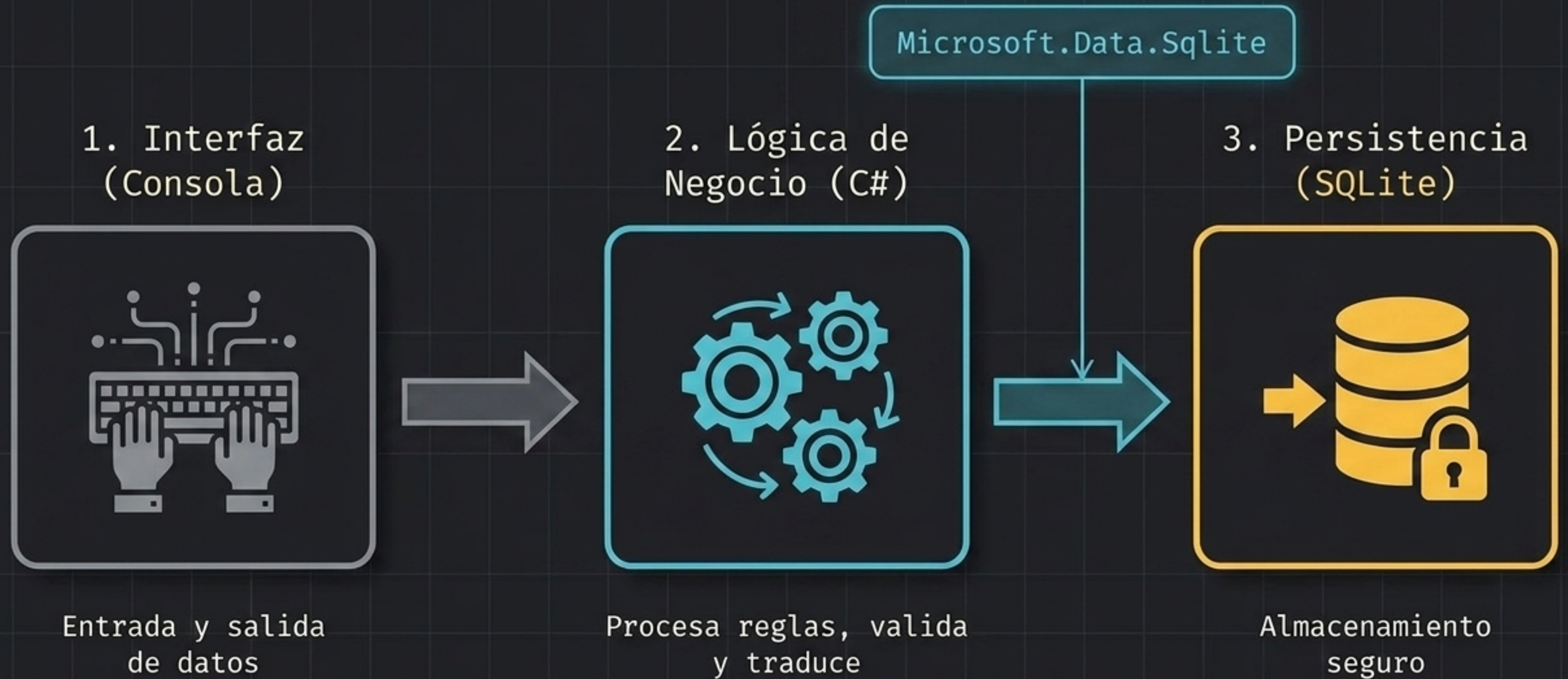


Usar un fichero SQLite (`agenda.db`)

➤ Persistente y ligero. Los contactos sobreviven al cierre de la aplicación.

SQLite es perfecto para este proyecto porque no requiere instalar un servidor pesado; toda tu base de datos vivirá en un simple fichero local.

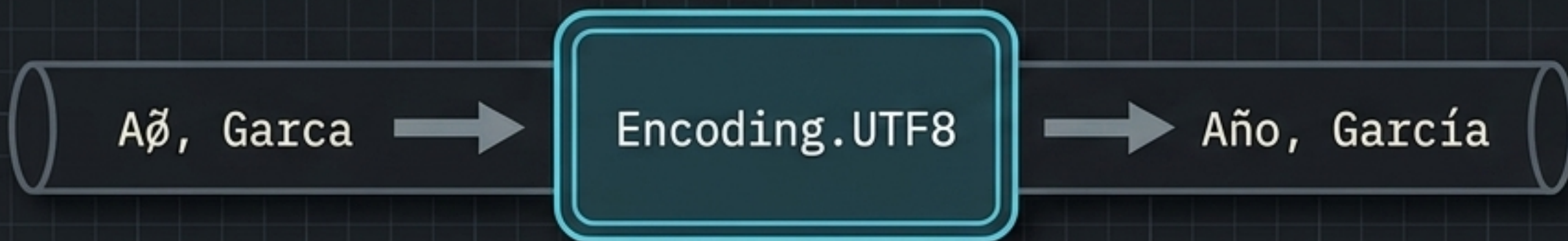
La Arquitectura de Nuestra Agenda



Preparando el Terreno: Comprendiendo el Español

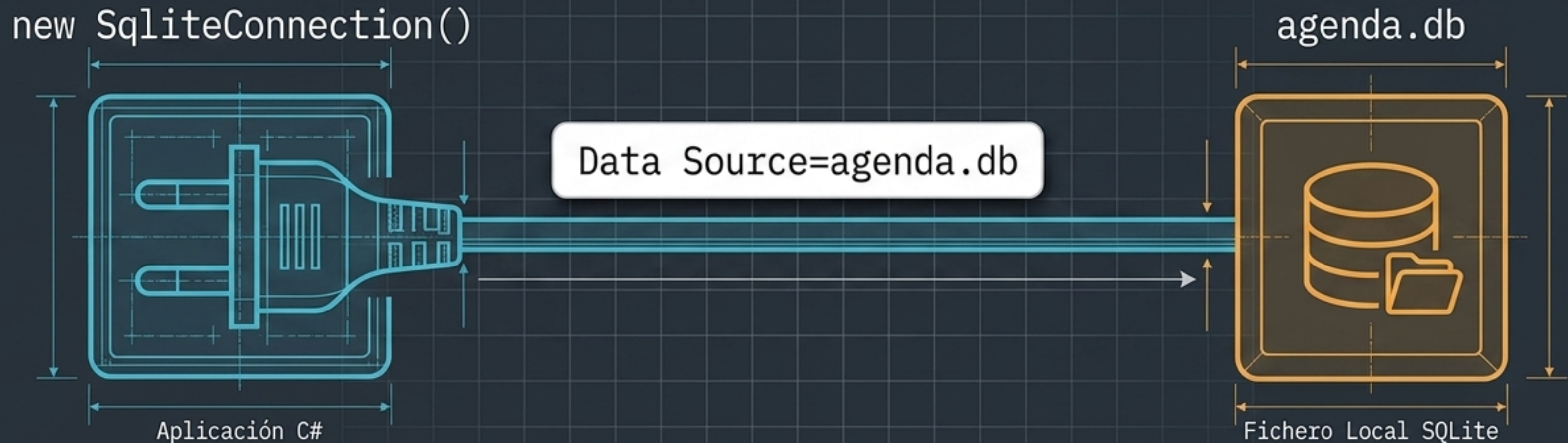
```
using System;
using System.Text;
using Microsoft.Data.Sqlite;

class ProgramaAgenda {
    static void Main() {
        Console.OutputEncoding = Encoding.UTF8;
    }
}
```



Crucial para el español de España: sin esta línea, la consola de Windows no entenderá nuestras tildes ni la 'ñ', mostrando símbolos extraños al leer la base de datos.

El Puente de Comunicación



Para que C# pueda hablar con SQLite, necesitamos establecer un canal. La cadena de conexión le dice a nuestro programa exactamente dónde encontrar el fichero de la base de datos en nuestro ordenador.

Diseñando el Esquema: La Tabla Contactos

Tabla: Contactos		
	Id	INTEGER PRIMARY KEY AUTOINCREMENT
	Nombre	TEXT
	Telefono	TEXT
	Email	TEXT

La Clave Primaria.

Es el identificador único de cada contacto. SQLite lo generará automáticamente, así que nunca tendremos dos personas mezcladas, incluso si se llaman igual.

El Ecosistema CRUD

Crear

INSERT → `ExecuteNonQuery()`

Leer

SELECT → `ExecuteReader()`

Actualizar

UPDATE → `ExecuteNonQuery()`

Destruir

DELETE → `ExecuteNonQuery()`

Todo lo que haces en una agenda se reduce a estos cuatro verbos.
Entender este mapa es la llave maestra para dominar las bases de datos desde C#.

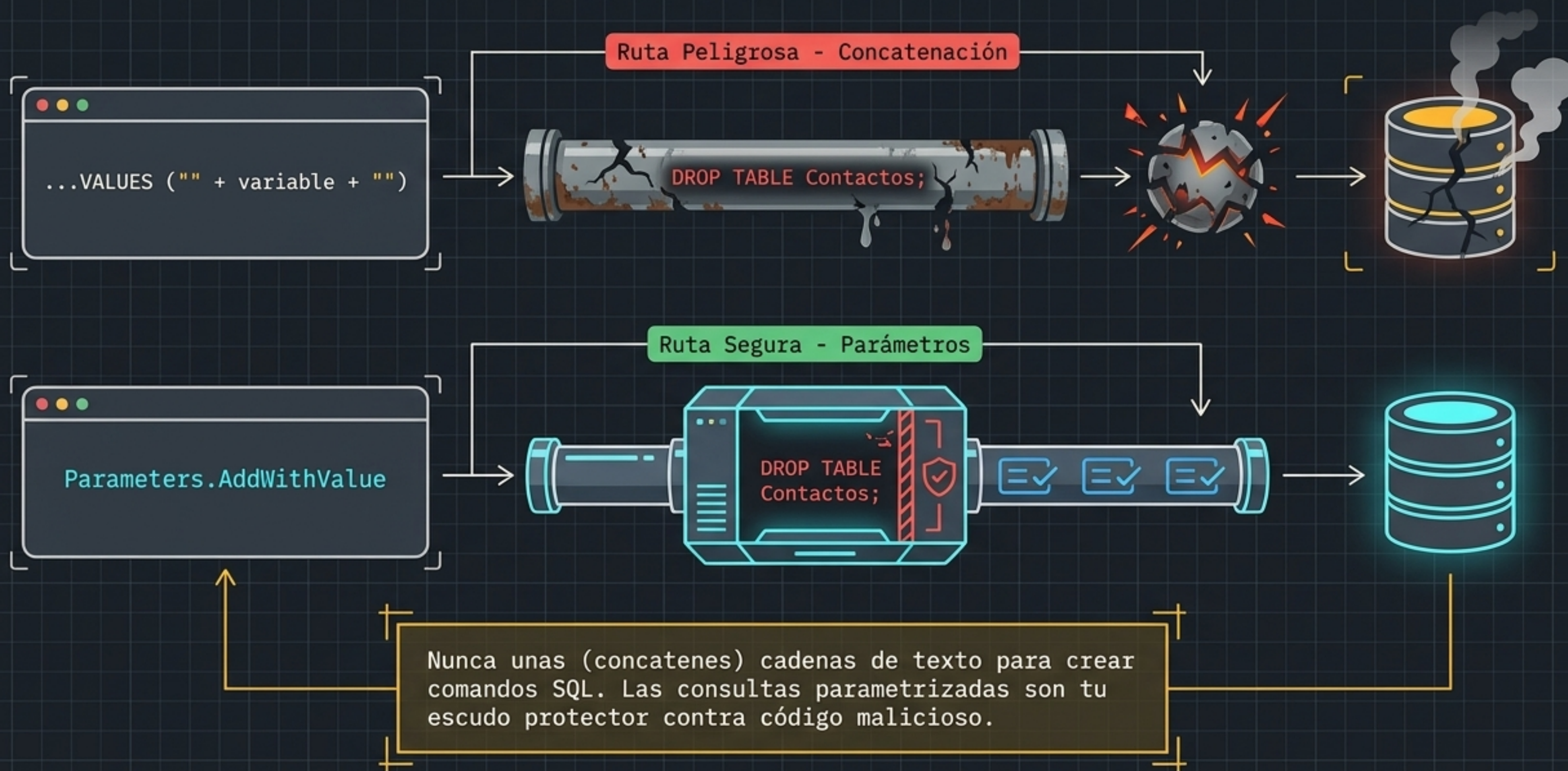
C – Añadiendo Contactos (Create)

```
string sql = "INSERT INTO Contactos (Nombre, Telefono) VALUES (@nombre, @tel)";  
var command = new SqlCommand(sql, connection);  
command.Parameters.AddWithValue("@nombre", "María");  
command.Parameters.AddWithValue("@tel", "600123456");  
command.ExecuteNonQuery();
```

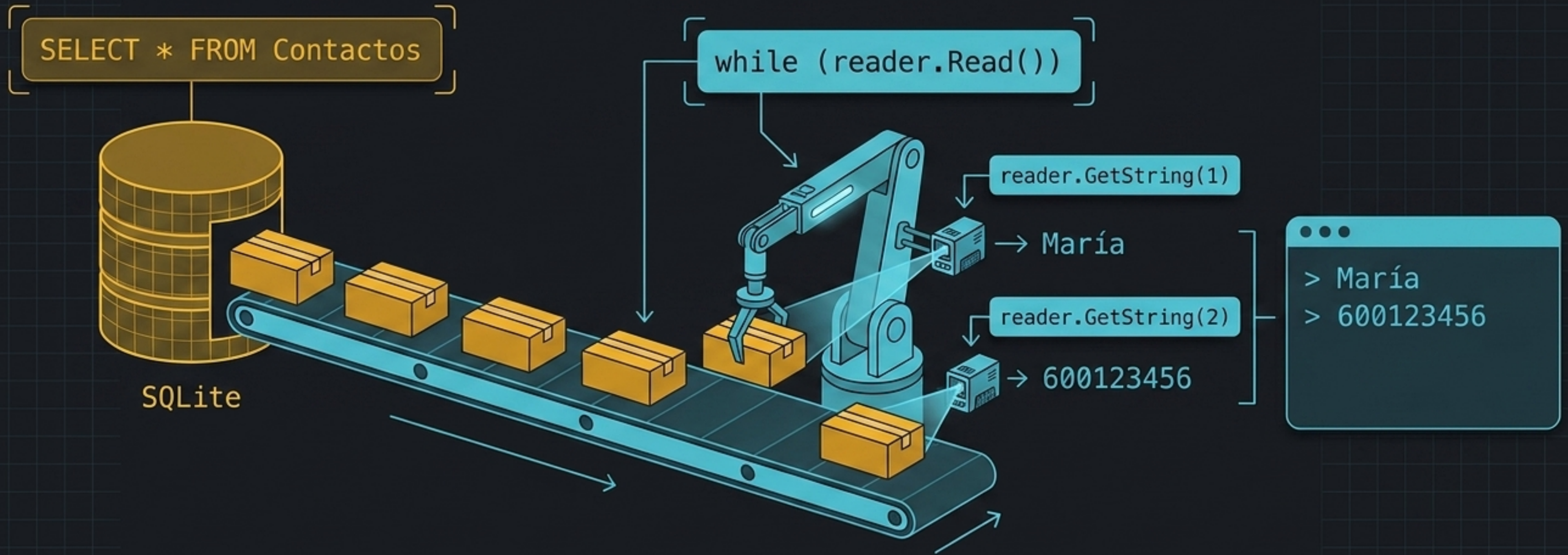


Ejecuta la acción y devuelve el número de filas afectadas en el fichero local.

El Escudo Anti-Hackers

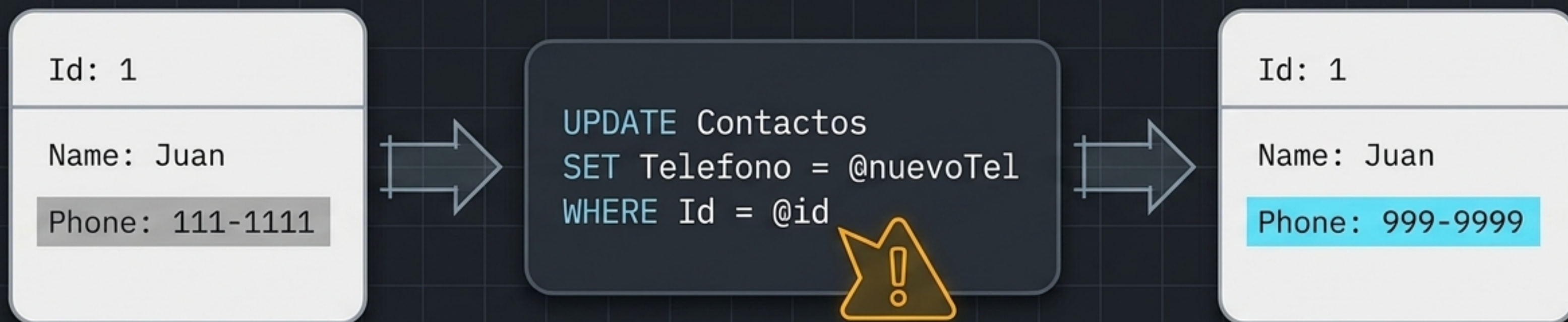


R - Leyendo la Agenda (Read)



El DataReader no carga toda la agenda de golpe.
Lee fila por fila, ahorrando memoria en tu ordenador.

U - Modificando Registros (Update)



¡Peligro! Sin la cláusula WHERE, el comando actualizará absolutamente todos los teléfonos de tu agenda al mismo número. El Id es tu mecanismo de precisión.

D - Limpiando la Agenda (Delete)



Consola

El usuario introduce el ID del contacto a eliminar.



C# Logic

El código prepara y protege el comando.

```
DELETE FROM Contactos WHERE Id = @id
```

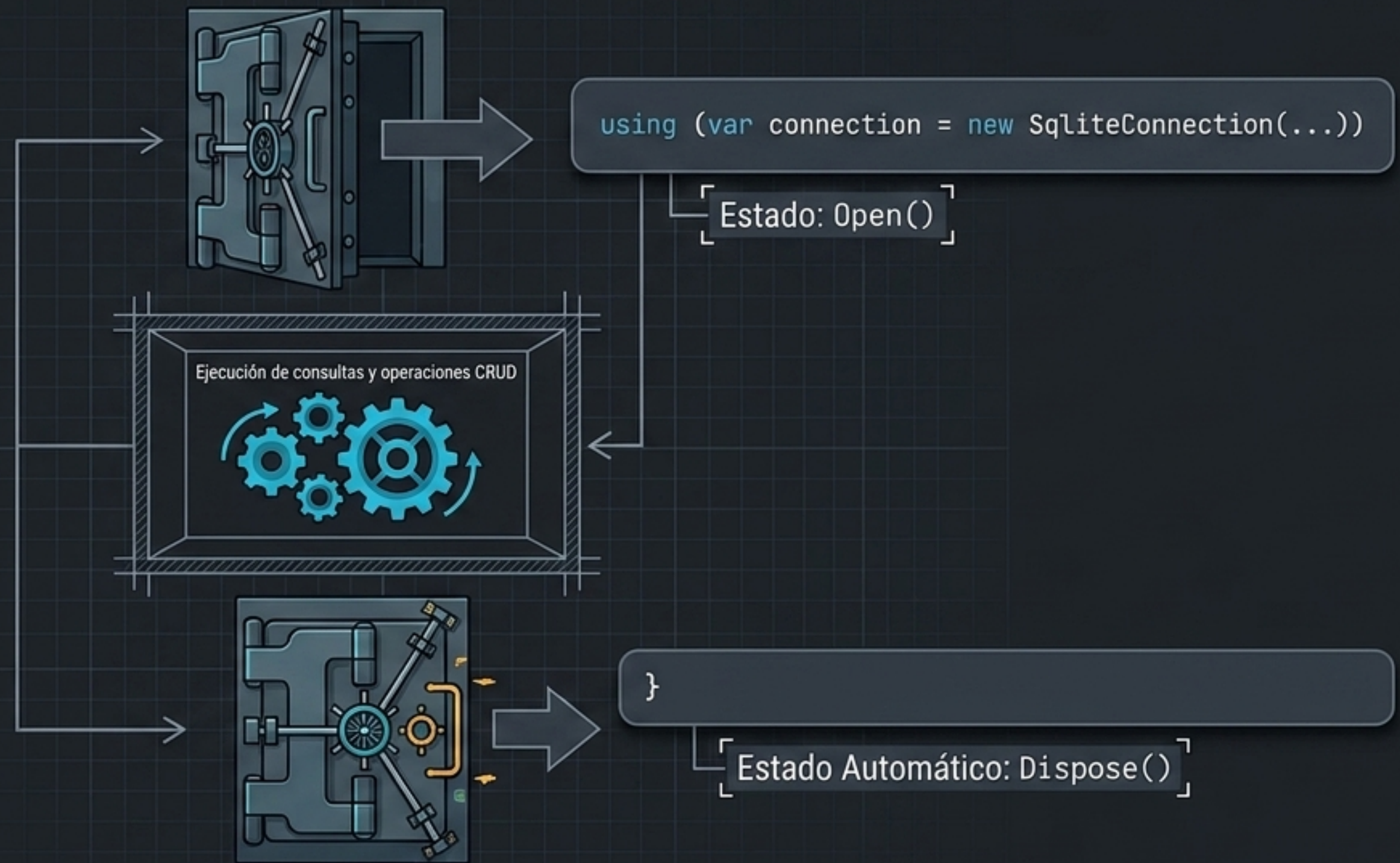


SQLite

La fila específica se borra permanentemente del fichero `agenda.db`.

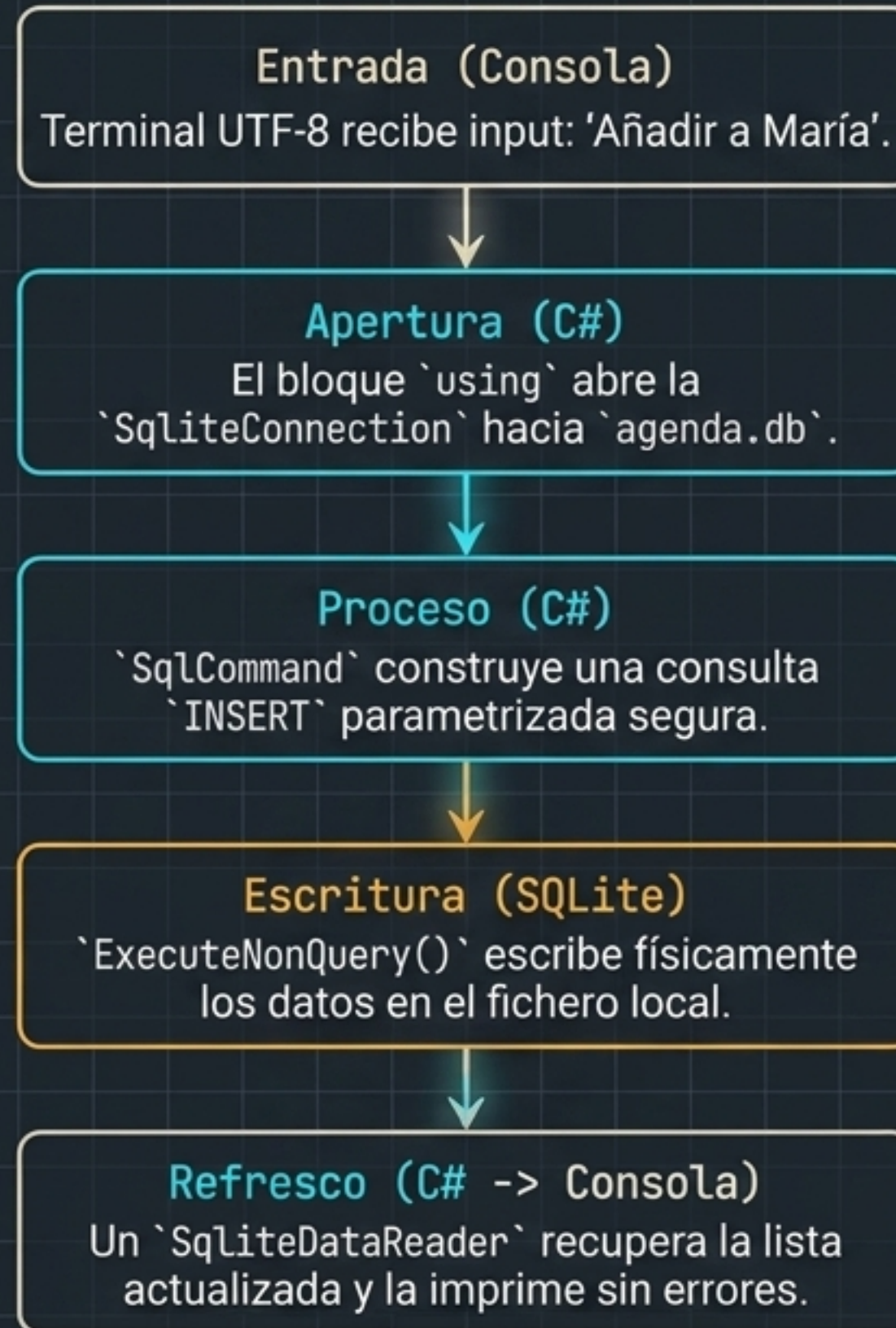
Al igual que con UPDATE, la eliminación debe ser quirúrgica. Usa siempre el Id proporcionado por el usuario como un parámetro protegido.

Gestión de Recursos: La Regla de Oro



El bloque `using` es un cierre de seguridad automático. Garantiza que el fichero de la base de datos se cierre y se libere la memoria, incluso si tu programa sufre un error inesperado.

Síntesis: El Flujo Vital de la Agenda



De una pulsación de tecla a
almacenamiento persistente.
Ahora comprendes cada
engranaje de la máquina.

Tu Turno: Próximos Desafíos



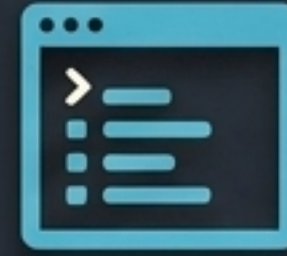
Búsqueda

Implementa la cláusula **LIKE** en SQL para buscar contactos por nombre.



Exportación

Usa `StreamWriter` en C# para exportar tu tabla a un fichero `.csv`.



Interfaz

Diseña un bucle `while` en la consola que muestre un menú interactivo permanente.

Ya tienes el motor funcionando. Ahora, construye el coche.